

FutureKawa

Référence API — Endpoints détaillés

Équipe : Rudy · Florian · Noé · Josselin

Version : 1.0

Date : 05/07/2026

Projet : FutureKawa

Sommaire

Référence API — FutureKawa

01 — Vue d'ensemble & conventions

02 — Authentification

03 — API Exploitation — endpoints

04 — API Siège — endpoints

05 — Codes de statut & erreurs

06 — Documentation OpenAPI

Référence API — FutureKawa

Projet	FutureKawa
Livrable	Référence API (endpoints détaillés)
Périmètre	Les deux API REST : Exploitation et Siège
Framework	Django REST Framework
Format	JSON
Doc vivante	OpenAPI / Swagger (/api/docs/)
Équipe	Rudy · Florian · Noé · Josselin

Sommaire

Section	Fichier
Vue d'ensemble & conventions	01-vue-ensemble.md
Authentification	02-authentification.md
API Exploitation — endpoints	03-api-exploitation.md
API Siège — endpoints	04-api-siege.md
Codes de statut & erreurs	05-codes-erreurs.md
Documentation OpenAPI	06-openapi-swagger.md

Deux API distinctes

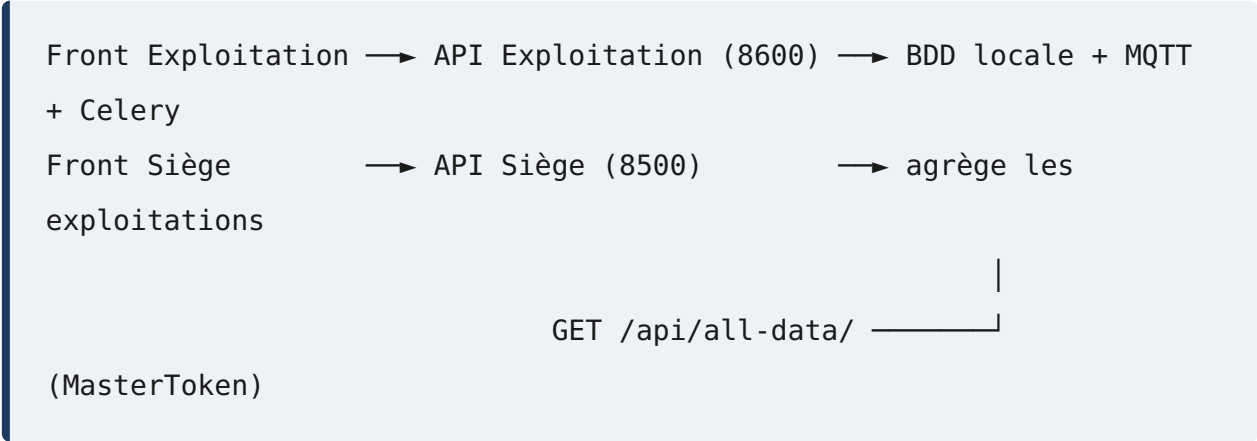
API	Base URL (démon)	Rôle
Exploitation	<code>http://localhost:8600/api</code>	Données terrain d'une plantation
Siège	<code>http://localhost:8500/api</code>	Agrégation multi-plantations

Chaque API est décrite dans le détail des livrables Backend correspondants ; ce livrable-ci est la **référence unifiée des endpoints**.

01 — Vue d'ensemble & conventions

Deux API REST

FutureKawa expose deux API construites avec **Django REST Framework**, renvoyant du **JSON**.



API	Base URL (démo)	Auth utilisateur
Exploitation	http://localhost:8600/api	JWT natif (Bearer)
Siège	http://localhost:8500/api	Token DRF (Token)

Conventions REST

Les ressources CRUD suivent les conventions standard de DRF (routeur `DefaultRouter`) :

Méthode	URL	Action
GET	/api/ressource/	Liste
POST	/api/ressource/	Création
GET	/api/ressource/{id}/	Détail
PUT / PATCH	/api/ressource/{id}/	Mise à jour (totale / partielle)

Méthode	URL	Action
DELETE	/api/ressource/{id}/	Suppression

Format des requêtes

```
POST /api/lots/ HTTP/1.1
Host: localhost:8600
Content-Type: application/json
Authorization: Bearer <jeton>

{ "champ": "valeur" }
```

En-tête	Rôle
Content-Type: application/json	Corps en JSON
Authorization	Jeton d'authentification (voir Authentification)

Format des réponses

Réponse type (collection) :

```
[
  { "id": 1, "name": "Entrepôt Nord", ... },
  { "id": 2, "name": "Entrepôt Sud", ... }
]
```

Réponse type (erreur) :

```
{ "error": "Message explicatif." }
```

Les codes de statut HTTP sont détaillés dans [Codes de statut & erreurs](#).

Identifiants des ressources

Ressource	Type d'identifiant
Lots (<code>batch</code>)	UUID (v4)
Autres ressources métier	Entier auto-incrémenté

Documentation interactive

Chaque API génère sa doc OpenAPI (Swagger / ReDoc) — utile pour explorer et tester en direct :

URL	Contenu
<code>/api/schema/</code>	Schéma OpenAPI (YAML)
<code>/api/docs/</code>	Swagger interactif
<code>/api/redoc/</code>	ReDoc

Voir [Documentation OpenAPI](#).

02 — Authentification

Trois mécanismes selon le contexte

Mécanisme	API	Usage
JWT natif (<code>Bearer</code>)	Exploitation	Authentification des utilisateurs exploitation
Token DRF (<code>Token</code>)	Siège	Authentification des utilisateurs siège
MasterToken (<code>Token</code>)	Exploitation	Accès du siège à <code>/api/all-data/</code>

Le détail sécurité (génération, révocation, chiffrement) est dans le livrable [Sécurité](#).

1. Exploitation — JWT natif

Connexion

```
POST /api/auth/login/
Content-Type: application/json

{ "email": "admin@futurekawa.com", "password": "admin" }
```

Le champ peut être `email` **ou** `username` .

Réponse

```
{
  "access": "eyJhbGciOi...",
  "refresh": "eyJhbGciOi..."
}
```



```
"user": { "email": "admin@futurekawa.com", "role": "admin" }
}
```

En complément, un cookie **HttpOnly** `session_token` est déposé (sécurisé hors debug, `SameSite=Lax`).

Utilisation

```
GET /api/lots/
Authorization: Bearer eyJhbGciOi...
```

Propriété du JWT	Valeur
Algorithme	HMAC-SHA256
Signature	avec <code>SECRET_KEY</code> Django
Expiration	24 h (<code>exp</code>)
Transport	en-tête <code>Authorization: Bearer</code> ou cookie <code>session_token</code>

2. Siège — Token DRF

Connexion

```
POST /api/auth/login/
Content-Type: application/json

{ "email": "admin@futurekawa.com", "password": "admin" }
```

Réponse

```
{
  "token": "9a8b7c6d5e4f3a2b1c...",
  "user": { "id": 1, "email": "...", "role": "admin" }
}
```

Utilisation

```
GET /api/sites/  
Authorization: Token 9a8b7c6d5e4f3a2b1c...
```

Déconnexion

```
POST /api/auth/logout/  
Authorization: Token 9a8b7c6d5e4f3a2b1c...
```

Supprime le token côté serveur.

3. MasterToken — accès inter-service

L'endpoint `/api/all-data/` de l'exploitation n'est **pas** accessible aux utilisateurs : il est réservé au siège, via un **token maître**.

```
GET /api/all-data/  
Authorization: Token a3f9b2c1d4e5f6... (64 hex = 256 bits)
```

Caractéristique	Détail
Génération	<code>python manage.py create_master_token "Siege Paris"</code>
Stockage	haché SHA-256 en base (<code>master_token</code>)
Révocation	<code>create_master_token ... --revoke</code> ou admin Django
Sécurité	256 bits d'entropie (<code>secrets.token_hex(32)</code>)

Le siège stocke ce token **chiffré** (Fernet) dans
`ExploitationSite.api_token` .

Définir son mot de passe (invitation)

Nouveau compte → email d'invitation avec lien à usage unique (24 h) :

```
POST /api/auth/set-password/  
Content-Type: application/json  
  
{ "token": "<uuid-invitation>", "password": "<nouveau-mot-de-  
passe>" }
```

Le token d'invitation est **supprimé après usage**.

Récapitulatif des en-têtes

API / cible	En-tête
Exploitation (utilisateur)	Authorization: Bearer <jwt>
Siège (utilisateur)	Authorization: Token <clé>
Exploitation <code>/api/all-data/</code> (siège)	Authorization: Token <master-token>

03 — API Exploitation — endpoints

Base URL (d  mo) : `http://localhost:8600/api` **Auth utilisateur :**
`Authorization: Bearer <jwt>` (voir [Authentification](#))

Syst  me & sant  

M��thode	Endpoint	Auth	R��le
GET	/api/	—	Tableau de bord de l'API
GET	/api/hello/	—	Test de connectivit��
GET	/api/health/	—	Sonde de sant�� (monitoring / CI)

```
GET /api/health/  
{ "status": "ok" }
```

Authentification

M��thode	Endpoint	Auth	R��le
POST	/api/auth/login/	—	Connexion → JWT
POST	/api/auth/set-password/	—	Mot de passe via token d'invitation

Entrep  ts, zones & capteurs

M��thode	Endpoint	Ressource
GET / POST	/api/entrepots/	Entrep��ts
GET / PUT / DELETE	/api/entrepots/{id}/	Un entrep��t
GET / POST	/api/zones/	Zones / parcelles

Méthode	Endpoint	Ressource
GET / POST	/api/sensors/	Capteurs
GET	/api/sensors/{id}/ history/	Historique des mesures d'un capteur

```
GET /api/entrepots/
[
  { "id": 1, "name": "Entrepôt Nord", "location": "São Paulo",
    "zones": 4 }
]
```

Capteurs — mesures en temps réel

Méthode	Endpoint	Rôle
GET	/api/capteur/latest/	Dernières mesures de chaque capteur
GET	/api/capteur/history/	Historique des mesures
GET	/api/capteur/stats/	Statistiques agrégées (min / max / moyenne)

```
GET /api/capteur/latest/
[
  {
    "sensor_id": 12,
    "warehouse": "Entrepôt Nord",
    "temperature": 24.7,
    "humidity": 61.2,
    "measured_at": "2026-07-05T14:32:10Z"
  }
]
```

Lots & mouvements

Méthode	Endpoint	Ressource
GET / POST	/api/lots/	Lots de café
GET / PUT / DELETE	/api/lots/{uuid}/	Un lot (identifiant UUID)
GET	/api/lots/{uuid}/history/	Historique des mouvements d'un lot
GET / POST	/api/batches/	Lots (vue technique)

```
GET /api/lots/{uuid}/history/
[
  { "date": "2026-07-01T09:00:00Z", "type": "entree", "zone": "Zone A" },
  { "date": "2026-07-03T16:20:00Z", "type": "transfert", "zone": "Zone B" }
]
```

Alertes & incidents

Méthode	Endpoint	Ressource
GET / POST	/api/alerts/	Alertes
GET / POST	/api/incidents/	Incidents déclarés
GET / POST	/api/incident-types/	Types d'incidents

```
GET /api/alerts/
[
  {
    "id": 87,
    "type": "temperature_critique",
    "warehouse": "Entrepôt Nord",
    "value": 32.4,
    "threshold": 28,
    "created_at": "2026-07-05T14:32:11Z"
  }
]
```

```
}  
]
```

Utilisateurs, rôles & configuration

Méthode	Endpoint	Rôle
GET / POST	/api/users/	Utilisateurs
GET	/api/page-permissions/	Permissions d'accès par page/rôle
GET / PUT	/api/parameters/	Paramètres globaux de l'exploitation
GET	/api/all/	Vue globale consolidée

Endpoint inter-service : /api/all-data/

Méthode	Endpoint	Auth
GET	/api/all-data/	MasterToken (Authorization: Token <master>)

Réponse : snapshot complet de la plantation (entrepôts, zones, capteurs, lots, mesures, alertes, incidents), consommé par le siège.

```
GET /api/all-data/  
{  
  "warehouses": [ ... ],  
  "sensors":    [ ... ],  
  "measures":   [ ... ],  
  "batches":    [ ... ],  
  "alerts":     [ ... ],  
  "incidents":  [ ... ]  
}
```

Réservé au siège ; inaccessible aux utilisateurs. Voir [Sécurité](#).

04 — API Siège — endpoints

Base URL (d  mo) : `http://localhost:8500/api` **Auth utilisateur :**

`Authorization: Token <cl  >` (voir [Authentification](#))

Syst  me & sant  

M��thode	Endpoint	Auth	R��le
GET	<code>/api/</code>	—	Message d'accueil
GET	<code>/api/health/</code>	—	Sonde de sant��

Authentification

M��thode	Endpoint	Auth	R��le
POST	<code>/api/auth/login/</code>	—	Connexion → token DRF
POST	<code>/api/auth/logout/</code>	Token	D��connexion (supprime le token)
POST	<code>/api/auth/set-password/</code>	—	Mot de passe via token d'invitation

```
POST /api/auth/login/
{ "email": "admin@futurekawa.com", "password": "admin" }
→
{ "token": "9a8b7c6d...", "user": { "id": 1, "role": "admin" } }
```

Sites d'exploitation

M��thode	Endpoint	R��le
GET	<code>/api/sites/</code>	Liste des plantations r��f��renc��es

Méthode	Endpoint	Rôle
POST	/api/sites/	Ajouter un site (token chiffré)
GET	/api/sites/{id}/	Détail d'un site
PUT / PATCH	/api/sites/{id}/	Modifier un site
DELETE	/api/sites/{id}/	Supprimer un site

Filtres (django-filter)

```
GET /api/sites/?country=Colombie&status=active
```

Création

```
POST /api/sites/
{
  "name": "Plantation Andes",
  "country": "Colombie",
  "api_url": "https://exploit-andes.futurekawa.com",
  "api_token": "a3f9b2c1d4e5f6..." // chiffré (Fernet) avant
stockage
}
→
{ "id": 4, "name": "Plantation Andes", "status": "unknown",
  "last_sync": null }
```

Le champ `api_token` n'est **jamais renvoyé** dans les réponses.

Actions de synchronisation (proxy vers l'exploitation)

Méthode	Endpoint	Rôle
POST	/api/sites/{id}/sync/	Appelle <code>/api/all-data/</code> du site et met à jour le cache

Méthode	Endpoint	Rôle
GET	/api/sites/{id}/lots/{batch_id}/history/	Proxy : historique d'un lot du site
GET	/api/sites/{id}/sensors/{sensor_id}/history/	Proxy : historique d'un capteur du site

Réponse de `sync`

```
POST /api/sites/4/sync/
{
  "status": "active",
  "last_sync": "2026-07-05T14:40:00Z",
  "data": { "warehouses": [...], "measures": [...] },
  "from_cache": false
}
```

En cas d'exploitation injoignable, le siège renvoie le **dernier cache connu** :

```
{
  "status": "unreachable",
  "last_sync": "2026-07-01T14:23:00Z",
  "data": { ... },
  "from_cache": true
}
```

Statut du site	Signification
<code>unknown</code>	Jamais synchronisé
<code>active</code>	Dernière sync réussie
<code>error</code>	L'API a répondu une erreur
<code>unreachable</code>	Site injoignable (timeout / réseau)

Utilisateurs du siège

Méthode	Endpoint	Rôle
GET	/api/users/	Liste des utilisateurs
POST	/api/users/	Créer un utilisateur + email d'invitation
GET / PUT / DELETE	/api/users/{id}/	Détail / modifier / supprimer
POST	/api/users/{id}/ resend-invitation/	Régénérer un token d'invitation (24 h)

```
POST /api/users/
{ "email": "op@futurekawa.com", "role": "viewer" }
→ utilisateur créé, email d'invitation envoyé
```

Rôle	Droits
admin	Lecture + écriture (sites, utilisateurs)
viewer	Lecture seule

05 — Codes de statut & erreurs

Codes de statut HTTP

Succès (2xx)

Code	Signification	Quand
200 OK	Requête réussie	GET, PUT, PATCH, actions
201 Created	Ressource créée	POST de création
204 No Content	Réussi, pas de corps	DELETE

Erreurs client (4xx)

Code	Signification	Cause typique
400 Bad Request	Requête invalide	Champ manquant / mal formé
401 Unauthorized	Non authentifié	Jeton absent, invalide ou expiré
403 Forbidden	Interdit	Rôle insuffisant, MasterToken révoqué
404 Not Found	Ressource introuvable	ID inexistant
405 Method Not Allowed	Méthode non permise	Ex. DELETE sur endpoint en lecture seule
429 Too Many Requests	Trop de requêtes	Throttle (ex. login siège)

Erreurs serveur (5xx)

Code	Signification	Cause typique
500 Internal Server Error	Erreur applicative	Bug, exception non gérée
503 Service Unavailable	Service indisponible	Exploitation injoignable (sync)

Format des erreurs

Les erreurs renvoient un objet JSON explicite :

```
{ "error": "Les champs identifiant et password sont obligatoires." }
```

Pour les erreurs de validation DRF (champ par champ) :

```
{  
  "email": ["Ce champ est obligatoire."],  
  "api_url": ["Entrez une URL valide."]  
}
```

Exemples concrets

Jeton manquant ou expiré

```
GET /api/lots/          (sans Authorization)  
→ 401 Unauthorized  
{ "detail": "Informations d'authentification non fournies." }
```

Identifiants invalides

```
POST /api/auth/login/  { "email": "x", "password": "mauvais" }  
→ 401 Unauthorized  
{ "error": "Identifiants invalides." }
```

MasterToken révoqué

```
GET /api/all-data/      Authorization: Token <révoqué>  
→ 403 Forbidden  
{ "detail": "Token maître invalide ou révoqué." }
```

Rôle insuffisant

```
DELETE /api/sites/4/      (utilisateur viewer)
→ 403 Forbidden
```

Exploitation injoignable (sync siège)

```
POST /api/sites/4/sync/   (exploitation down)
→ 200 OK avec le cache, ou 503 si jamais synchronisé
{ "status": "unreachable", "from_cache": true, ... }
```

Bonnes pratiques côté client

Situation	Réaction recommandée
401	Rediriger vers la connexion, rafraîchir le jeton
403	Afficher « accès refusé »
429	Attendre avant de réessayer (backoff)
503 / from_cache: true	Afficher un badge « données en cache »
5xx	Message générique + journalisation

06 — Documentation OpenAPI

Une documentation qui se génère seule

Les deux API utilisent **drf-spectacular** pour produire automatiquement leur documentation au format **OpenAPI 3**. Elle est donc toujours **à jour avec le code** : chaque endpoint, paramètre et schéma est extrait des ViewSets et serializers.

Les trois points d'accès

URL	Contenu	Usage
<code>/api/schema/</code>	Schéma OpenAPI brut (YAML)	Import dans des outils (Postman, générateurs de clients)
<code>/api/docs/</code>	Interface Swagger UI	Explorer et tester les endpoints en direct
<code>/api/redoc/</code>	Interface ReDoc	Lecture élégante, orientée documentation

Exemples (démon) :

```
Exploitation : http://localhost:8600/api/docs/  
Siège       : http://localhost:8500/api/docs/
```

Swagger UI — tester en direct

Swagger UI permet d'appeler l'API depuis le navigateur :

1. Ouvrir `/api/docs/`
2. Cliquer « Authorize » et coller son jeton
 - Exploitation : Bearer <jwt>
 - Siège : Token <clé>
3. Déplier un endpoint → « Try it out » → renseigner les paramètres →

« Execute »

4. Lire la réponse réelle (statut, corps JSON, en-têtes)

Enrichissement de la doc

Les vues sont annotées pour une documentation plus lisible (`drf-spectacular`) :

```
@extend_schema_view(
    list=extend_schema(summary="Lister les incidents",
                        description="Récupère la liste complète des incidents/alertes."),
    create=extend_schema(summary="Créer un incident"),
)
class AlertViewSet(viewsets.ModelViewSet):
    ...
```

Ces annotations alimentent les résumés et descriptions visibles dans Swagger / ReDoc.

Générer un client à partir du schéma

Le schéma OpenAPI (`/api/schema/`) peut servir à générer automatiquement du code client :

Outil	Usage
Swagger Codegen / OpenAPI Generator	Générer un SDK (TS, Python, ...)
Postman	Importer la collection pour tester
Insomnia	Import du schéma pour explorer

Rappel : ce livrable vs la doc vivante

	Ce livrable (Référence API)	Swagger <code>/api/docs/</code>
Nature	Document figé, pédagogique	Interface interactive, live

	Ce livrable (Référence API)	Swagger /api/docs/
Avantage	Vue d'ensemble, exemples commentés	Toujours exact, testable
À utiliser pour	Comprendre l'architecture des endpoints	Essayer une requête précise

Les deux sont complémentaires : commencer par ce livrable pour la vue d'ensemble, puis Swagger pour l'exploration fine.